

TASK 1

• JDK、JRE、JVM

◦ JVM

JVM, 全称**Java Virtual Machine**, 中文**Java虚拟机**, 负责运行java程序。

java程序写好并编译后会产生 `.class` 这一含java字节码的二进制文件, 而这个格式的文件给到jvm后, jvm会将其转为机器码以在计算机硬件运行

JVM在不同系统有不同版本, 但是.class文件与平台没有关系, 这使同一份源码的.class文件在各个系统上都可以运行, 让语言不需要针对不同硬件来重新编译源码, 生成不同的机器码

◦ JRE

JRE, 全称**Java Runtime Environment**, 中文**Java运行环境**, 用于执行已经完成编译的Java程序 (没有开发和编译能力)

JRE主要包含了JVM和Java标准类库两个部分 (jvm见上)

java标准类库, 又叫Java API, 里面存有预先编写好的类 (代码) 和接口 (功能规范) 的汇总。处理字符串、随机数等 等常见的功能可以从此直接调用

JRE可以直接运行编译完成的 `.class` 文件, 也可以运行打包完成的 `.jar` 应用程序, 并提供运行必须的内存管理等支持。

总结: JRE=JVM+库, JRE主要作用为运行

◦ JDK

JDK, 全称**Java Development Kit**, 中文**Java开发工具包**, 用于编写、编译、调试Java程序等。

JDK主要包含JRE和开发工具集两个部分

■ JRE:

见上

■ 开发工具集

jdk包含的基本组件很多, 最主要的两个是 `javac` 和 `java`。

`javac`, 是java编译器, 负责将程序转换为字节码

`java`, 负责启动jvm, 执行编译好的.class程序

还有其他的调试工具、打包工具、文档生成器、查错工具等等

JDK=JRE+开发工具集

总结: 三者关系: JDK>JRE>JVM

• 为什么安装了这些工具之后Java就能正常编译运行了?

◦ 编译

JDK的 `javac` 编译器读取Java源码, 并把它翻译为jvm可以识别的.class字节码文件

◦ 运行

需要用到JDK内部的JRE(JRE内部有jvm和Java标准类库)

jvm读取.class字节码，并把它转为机器码，并借助Java API里预先写好的类代码功能使程序运行

我的JDK版本号：25.0.4.1 (JDK25)

java-version:

```
java version "25.0.4.1" 2026-08-18 LTS
Java(TM) SE Runtime Environment (build 25.0.4.1+1-LTS-5)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.4.1+1-LTS-5, mixed mode, sharing)
```

javac -version→javac 25.0.4.1

TASK 2

• 我配置的环境变量

- JAVA_HOME

存放JDK的安装根目录，里面存着我JDK的路径，帮助开发工具自动定位JDK

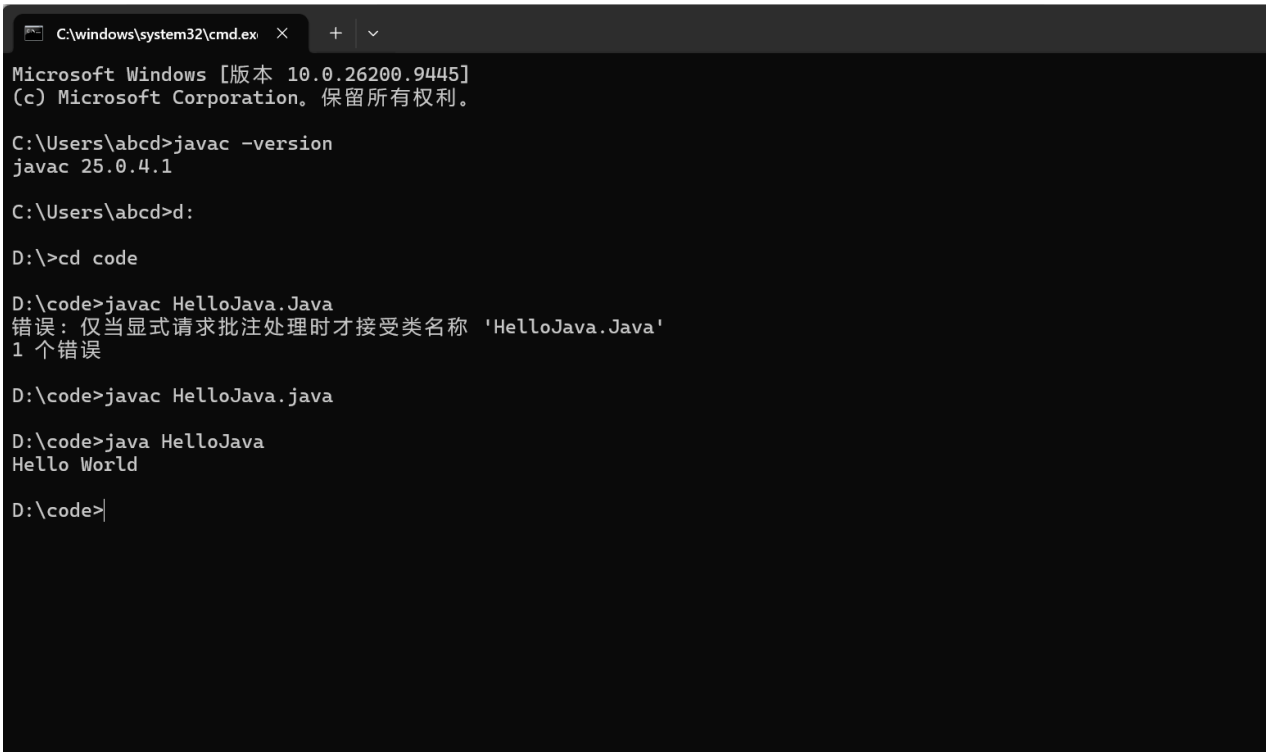
- Path

新增了%JAVA_HOME%\bin，使在任意终端目录都可以执行java和javac命令，否则仅限在bin里执行

• 为什么配置完成后，在命令行里就可以直接识别 java 和 javac

因为我在配置了path环境变量，把bin目录添加到了系统的搜索清单里，且bin里有java、javac等这些程序，这样在命令行敲这些命令时，系统会去path里查找调用，于是能直接识别到

• 运行 我第一次打成大写然后报错子



```
C:\windows\system32\cmd.exe
Microsoft Windows [版本 10.0.26200.9445]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\abcd>javac -version
javac 25.0.4.1

C:\Users\abcd>d:

D:\>cd code

D:\code>javac HelloJava.Java
错误: 仅当显式请求批注处理时才接受类名称 'HelloJava.Java'
1 个错误

D:\code>javac HelloJava.java

D:\code>java HelloJava
Hello World

D:\code>|
```

这里javac是编译命令，负责编译和查错，输出一个.class文件

TASK 3

• 这段代码可以大致分成哪几个部分？每个部分的作用是什么？

主要可以分4为个部分

- 第一部分： `package com.Example;`

是包的声明，表示后面的类HelloWorld和Test都在这个包内，且这两个类的全限定类名加上com.Example以便于区分管理

- 第二部分： `import com.Example.tool.Print;`

import引用这个包外的一个类“Print”这样以后使用print这个静态方法不需要写全限定类名

○

- 第三部分：

```
public class HelloWorld {  
    public static void main(String[] args) {  
        Test.test();  
    }  
}
```

类HelloWorld的内容，其中main是程序入口，里面调用了test这个静态方法进而输出helloworld

- 第四部分：

```
class Test {  
    public static void test() {  
        Print.print("Hello world");  
    }  
}
```

类Test的内容，在test这一静态方法中调用了print这个静态方法，当它被调用时，输出helloworld

• 什么是包package？为什么Java项目里经常需要包结构？

包是Java用来组织类的机制。

因为Java使用包结构可以更方便地组织、管理代码，提升代码的可维护性

可以通过包访问权限控制类的访问范围，比如一些类只能在包内访问

还可以生成全限定类名，避免类的名称冲突

- **import有什么用**

import导入，用来导入其他包的类，使得在这个代码内使用该类时不需要再使用这个类的全限定类名

- **main 方法为什么能作为程序入口？**

和JVM有关，JVM有固定的规则：程序启动后，会自动寻找 `public static void main(String[] args)` 的 main方法并执行，因此 main 方法作为程序入口。

TASK 4

- **项目结构**

1. src: java源码根目录
2. com.example:文件路径com/example，包
3. tool, com.example的子包
4. HelloWorld.java类在example包内，Print.java类在tool包内

Print.java

```
package com.Example.tool;

public class Print {
    public static void print(String str) {
        System.out.println(str);
    }
}
```

成功输出了 `Hello world!!`！

用IDEA和命令行有什么差异？emm目前只感觉到了idea管理很方便而且不用我手动去编译